

# Introdução

Notas sobre o livro: Algoritmos Genéticos, Teoria e implementação

## Algoritmos Evolucionários

Algoritmos evolucionários usam modelos computacionais dos processos naturais de evolução como ferramenta para resolver problemas.

Comportamento padrão dos algoritmos evolucionários:

```
t:=0 // Contador de tempo
Inicializa_População P(0) // Inicialização randômica

Enquanto não terminar faça // Condição de término: pro tempo/avaliação/etc
  Avalie_População P(t) // Avalia população no instante
  P' := Selezione_Pais P(t) // Seleciona sub-população que gerará nova geração
  P = Recombinação_e_mutação // Aplica operadores genéticos
  Avalie_População P' // Avalia nova população
  P(t+1) = Selezione_sobreviventes P(t), P' // Seleciona sobreviventes dessa geração
  t:=t+1 // Incrementa contador de tempo
Fim enquanto
```

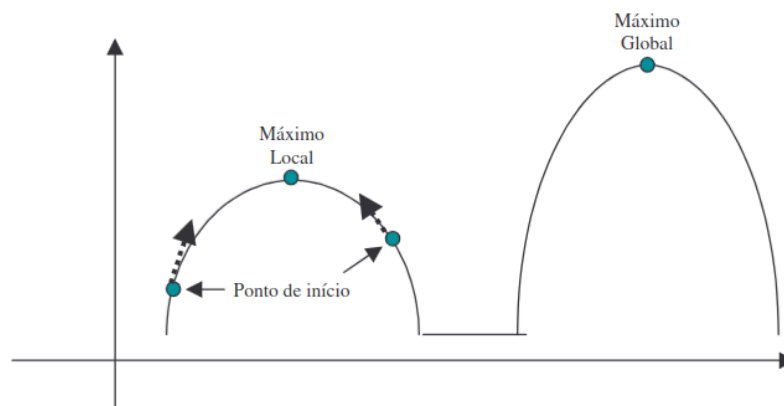
Algoritmos evolucionários são extremamente dependentes de fatores estocásticos (probabilísticos), tanto na fase de inicialização da população quanto na fase de evolução (durante a seleção dos pais, principalmente). Assim, eles são heurísticas (técnica que troca a exatidão e otimização por velocidade, mas não garante a solução perfeita ou ótima).

Se você tem um algoritmo com tempo de execução razoável para execução de um problema, então não há necessidade de se usar um algoritmo evolucionário. Sempre dê prioridade aos algoritmos exatos. Algoritmos evolucionários entram em cena para resolver problemas cujos algoritmos são extremamente lentos (problemas NP-completos) ou incapazes de obter solução (como maximização de funções multi-modais)

## Algoritmos Genéticos (GA)

São um ramo dos algoritmos evolucionários. São técnicas heurísticas de otimização global (algoritmo de busca de máximos cada vez melhores a cada execução que não garantem solução ótima) baseadas nos mecanismos de seleção natural e genética.

Se opõe a métodos como **hill climbing**, que seguem a derivada/gradiente de uma função tentando encontrar o máximo, ficando facilmente retidos em máximos locais. GAs não ficam estagnados ao encontrarem máximos locais.



No GA, codificamos a informação em cromossomos. Cada cromossomo representa um indivíduo e possui genes, onde cada gene será uma característica a ser avaliada (velocidade, altura, etc).

## O GA mais básico

### Esquema de um GA

1. Inicialize a população de cromossomos
2. Avalie cada cromossomo na população
3. Selecione os pais para gerar novos cromossomos. Aplique os operadores de recombinação e mutação a esses pais para gerar os novos indivíduos (nova geração)
4. Apague os velhos membros da população
5. Avalie todos os novos cromossomos e insira-os na população
6. Se o tempo acabou, ou o melhor cromossomo satisfaz os requerimentos e performance, retorne-o, caso contrário volte para 3)

### Representação cromossomial

A representação cromossomial denota um indivíduo. De modo básico, é uma estrutura com partições, que chamamos de genes. Os genes são os valores a serem alterados para buscar um máximo global. Por exemplo:

```
Cromossomo1 = [int: velocidade, int: distancia_de_salto]
```

Regras gerais:

1. A representação deve ser a mais simples possível
2. Se houver soluções proibidas ao problema, então não devem ter uma representação
3. Se o problema impuser condições de algum tipo, estas devem estar implícitas dentro da representação

### Exemplo: Mínimo de função

Seja o problema encontrar o mínimo da função no intervalo  $(x, y) \in [(-100, -100), (100, 100)]$ :

$$f(x, y) = 0.5 + \frac{\sin\left((x^2 + y^2)^{\frac{1}{2}}\right)^2 - 0.5}{(1 + 0.001(x^2 + y^2))^2}$$

Essa função conhecida como **Função de Schaffer N. 2**, uma função clássica de benchmark para algoritmos genéticos por ser altamente não-linear e multivariada, contendo muitos mínimos/máximos locais dispostos em anéis concêntricos ao redor da origem.

Para resolver esse problema, podemos usar uma representação binária de 44 bits (precisão que queremos pra solução), onde os primeiros 22 bits representam x e os outros 22 representam y. A representação binária é boa pois para ela os operadores genéticos são extremamente simples, mas existem outras representações.

22 bits nos dá números entre 0 e  $2^{23} - 1$ . Para colocá-los na faixa de -100 a 100, multiplicamos o número dado pelos bits por  $\frac{100}{2^{22}-1}$ . Assim, codificamos dois números (x,y) que podem ser aplicados à função de avaliação/fitting do problema.

```
// C++ Random Generator para uma boa distribuição probabilística
random_device dev;
mt19937 rng(dev());
uniform_int_distribution<int> dist1_int(0,1); // Inteiro no intervalo [0,1]
// Para gerar random: dist1_int(rng)

// Recebe string e a preenche com valores binários aleatórios
void inicializaElemento(string &cromossomo, int tamanho){
```

```

for(int i = 0; i < tamanho; i++){
    if(dist1_int(rng)){
        cromossomo.push_back('1');
    } else {
        cromossomo.push_back('0');
    }
}
}

```

## Função de avaliação/fitting

A função de avaliação é a ferramenta para os GAs determinarem a qualidade de um indivíduo como solução do problema.

Ela não é necessariamente uma função  $f: \mathbb{R}^n \rightarrow \mathbb{R}$ . Pode qualquer tipo de função, inclusive discreta. É importante que ela **tenha poder discriminativo sobre os indivíduos** de modo adequado para **selecionar os que melhor resolvem o problema**.

Nenhum elemento deve ter avaliação negativa ou zero. Isso faria com que a soma das avaliações diminuísse, alterando a roleta e criando uma distribuição errada. (método da roleta será explicado mais pra frente. Ele aumenta a chance de indivíduos com uma avaliação melhor serem selecionados).

Por exemplo: para 3 indivíduos avaliados com nota **1**, **-5** e **20**, o total é **16**. O indivíduo com avaliação 20 representaria uma proporção  $\frac{20}{16}$ , o que equivale a 125% da roleta/soma, tornando a distribuição entre 0% e 100% impossível.

Para resolver isso é simples. Se o mínimo global da função de avaliação é  $f(x_{\min}) = -c$ , basta utilizarmos a função de avaliação  $f'(x) = f(x) + c$ .

Outro ponto importante é que os GAs são técnicas de **maximização** apenas. Se quisermos priorizar indivíduos com avaliação menor, basta invertermos a função de avaliação:  $g(x) = \frac{1}{f(x)}$

Também devemos tratar os casos onde  $f(x) = 0$ .

### Exemplo (continuação): Mínimo de função

Uma possível implementação da função de avaliação para a função

$$f(x, y) = 0.5 + \frac{\sin\left((x^2 + y^2)^{\frac{1}{2}}\right)^2 - 0.5}{(1 + 0.001(x^2 + y^2))^2}$$

Obs: Como queremos encontrar o **mínimo**, vamos encontrar a função  $h(x) = \frac{1}{f(x)+c}$ , onde c é uma constante real positiva qualquer. (trocamos o problema de achar mínimo para achar um máximo)

Seria:

```

double binParaDouble(string &cromossomo, int inicio, int fim){
    double aux = 0;

    for(int i = 0; i <= (fim-inicio); i++){
        if(cromossomo[i+inicio] == '1') aux += pow(2,i);
    }

    return aux;
}

double calculaAvaliacao(string &cromossomo) {
    // (100.0 / (2^(22)-1)) = approx 0.000023841863594449
    double x = binParaDouble(cromossomo, 0, 21) * 0.000023841863594449;
    double y = binParaDouble(cromossomo, 22, 43) * 0.000023841863594449;
}

```

```
//cout << x << " " << y << endl;

double numerador = pow(sin(sqrt(x*x + y*y)), 2) - 0.5;
double denominador = pow((1.0 + 0.001*(x*x + y*y)), 2);
double avaliacao = 1.0/(0.5 + numerador / denominador); // Inverte para trocar mínimo pelo máximo

return avaliacao;
}
```

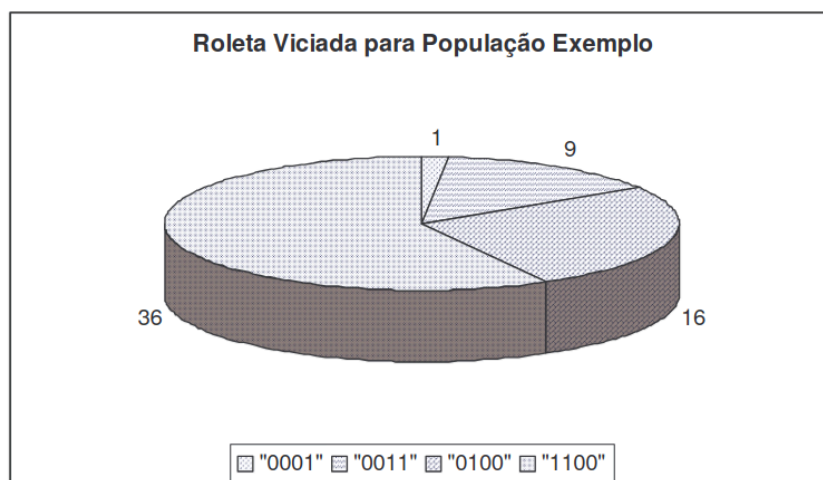
## Seleção de pais

O método de seleção de pais que utilizaremos deve tentar simular o mecanismo de seleção natural que atua sobre as espécies biológicas, em que os pais mais capazes geram mais filhos, mas os pais menos aptos também podem gerar descendentes. Assim, **temos que privilegiar os indivíduos com avaliação alta sem desprezar completamente aqueles com função de avaliação muito baixa.**

Isso ocorre pois até indivíduos com péssima avaliação podem ter características genéticas que sejam favoráveis à criação de um “superindivíduo”, então eles não podem ser completamente descartados.

Para fazer isso, usamos o método da **roleta viciada**, na qual cada cromossomo recebe um pedaço proporcional à sua avaliação.

| Indivíduo | Avaliação | Pedaço da roleta (%) |
|-----------|-----------|----------------------|
| 00001     | 1         | 1.61                 |
| 0011      | 9         | 14.41                |
| 0100      | 16        | 25.81                |
| 0110      | 36        | 58.07                |
| Total     | 62        | 100.00               |



Depois, rodamos a roleta (sorteia-se número entre 0 e 100) e isto seleciona o indivíduo/cromossomo.

Lembre-se que  $Total = \sum f_{X_i}$ . Se houver indivíduo com avaliação negativa, a soma dos espaços alocados para os de avaliação positiva excederia  $360^\circ$ . Por isso, evita-se uma função de avaliação/fitting eu tenha resultados negativos.

```
(a) Some todas as avaliações
(b) Ordene todos os indivíduos em ordem crescente de avaliação (opcional)
(c) Selecione um número s entre 0 e soma
(d) i=1
(e) aux = avaliação do indivíduo 1
(f) enquanto aux < s
(g)     i = i + 1
(h)     aux = aux + avaliação do indivíduo i
(i) fim enquanto
```

### Exemplo (continuação): Mínimo de função

```
int giraRoleta(vector<double> &avaliacao, int n, double soma_roleta){
    double aux = 0;
    double rnd = dist1_real(rng) * soma_roleta; // Evita operador / (divisão de ponto flutuante), que é
    custoso

    for(int i = 0; i < n; i++){
        aux += avaliacao[i];
        if(aux >= rnd) return i;
    }

    // Se o último for o escolhido (rnd = 90, last = 95)
    return n-1;
}
```

### Operador de crossover e mutação

Vamos começar com o operador de crossover mais simples, o single-point crossover. Outros operadores mais complexos (e mais eficientes) serão apresentados posteriormente.

Esse operador é simples. Depois de selecionados dois pais, um ponto de corte é selecionado. Um ponto de corte é uma posição entre dois genes de um cromossomo. Cada indivíduo de n genes possui n-1 pontos de corte (não necessariamente dividimos o cromossomo em partes iguais). O ponto de corte é selecionado por sorteio. No caso de apenas 2 genes, só há um ponto de corte.

O primeiro filho é composto através da concatenação da parte esquerda do primeiro pai com a parte direita do segundo pai. O segundo filho, com as partes que sobraram.

A metade à esquerda do ponto de corte de um pai vai para um filho e a metade à direita vai para outro. Isso é importante. Se criarmos apenas um filho, probabilisticamente podemos estar jogando alguma parte importante do cromossomo fora.

### Exemplo (continuação): Mínimo de função

No exemplo de mínimo de função, temos o cromossomo: 10000110011100101101110100101001100101100101, onde a primeira metade é x, e a segunda, y (os genes).

Aqui, **consideraremos cada bit como um gene**. Isso traz uma variabilidade maior, pois se apenas cortarmos ao meio e fazer o crossover com o outro pai, x e y permanecerão imutáveis (apenas trocamos). Para o algoritmo genético funcionar, precisamos ter variabilidade.

Assim, com 44 bits, temos 43 pontos de corte possíveis.

```
void singlePointCrossover(string &pai1, string &pai2, string &filho1, &filho2, int n){
    int rnd = dist43(rng);
```

```

for(int i = 0; i < rnd; i++){
    filho1.push_back(pai1[i]);
    filho2.push_back(pai2[i]);
}
for(int i = rnd; i < n; i++){
    filho1.push_back(pai2[i]);
    filho2.push_back(pai1[i]);
}
}

```

Essa abordagem funciona pois são selecionados os bits mais significativos (MSB) do primeiro vencedor com os menos significativos (LSB) do segundo vencedor.

Os MSB definem a macrorregião do plano, enquanto os LSB definem o ajuste fino local. Através das gerações, é esperado que os dois pais selecionados tenham bons MSB e LSB, que ao serem misturados, possuem chances e gerar um filho melhor que os dois.

Existem abordagens diferentes e mais eficientes que podem ser exploradas.

## Operador de mutação

Depois de formar o filho, para aumentar a variabilidade, aplicamos o operador de mutação. Ele tem associada a ele uma probabilidade muito baixa, chamada de taxa de mutação (da ordem de 0.5%) e sorteamos um valor entre 0 e 1. Se ele for menor que a taxa de mutação, altera-se o valor do gene aleatoriamente.

### Exemplo (continuação): Mínimo de função

```

void mutacao(string &filho, int n, double taxa_mutacao){
    for(int i = 0; i < n; i++){
        double rnd = dist1_real(rng);

        if(rnd < taxa_mutacao){
            // Inverte
            if(filho[i] == '1') filho[i] = '0';
            else filho[i] = '1';
        }
    }
}

```

## Módulo de população

Os pais têm que ser substituídos conforme os filhos vão nascendo, já que a população final deve possuir tamanho fixo sempre.

Sabemos que a cada atuação do nosso operador genético estamos criando dois filhos. Estes vão sendo armazenados em um espaço auxiliar até que o número de filhos criado seja igual ao tamanho da população.

Neste ponto o módulo de população entra em ação. Vamos usar um módulo de população simples: Descartar todos os pais, tornando os filhos a nova população.

### Exemplo (continuação): Mínimo de função - Algoritmo final

```

#include<string>
#include<vector>
#include<random>
#include<iostream>
#include<math.h>
#include<iomanip>
#include<algorithm> // Para sort()

```

```

using namespace std;

// C++ Random Generator para uma boa distribuição probabilística
random_device dev;
mt19937 rng(dev());
uniform_int_distribution<int> dist1_int(0,1); // Inteiro no intervalo [0,1]
uniform_int_distribution<int> dist43(1,43); // Inteiro no intervalo [1,43]
uniform_real_distribution<double> dist1_real(0.0, 1.0); // Double no intervalo [0, 100]
// Para gerar random: dist1(rng) // dist43(rng) // dist1(rng)

void inicializaElemento(string &cromossomo, int tamanho){
    for(int i = 0; i < tamanho; i++){
        if(dist1_int(rng)){
            cromossomo.push_back('1');
        } else {
            cromossomo.push_back('0');
        }
    }
}

double binParaDouble(string &cromossomo, int inicio, int fim){
    double aux = 0;

    for(int i = 0; i <= (fim-inicio); i++){
        if(cromossomo[i+inicio] == '1') aux += pow(2,i);
    }

    return aux;
}

double calculaAvaliacao(string &cromossomo) {
    // (100.0 / (2^(22)-1)) = approx 0.0000119209304
    double x = binParaDouble(cromossomo, 0, 21) * 0.000023841863594449;
    double y = binParaDouble(cromossomo, 22, 43) * 0.000023841863594449;

    //cout << x << " " << y << endl;

    double numerador = pow(sin(sqrt(x*x + y*y)), 2) - 0.5;
    double denominador = pow((1.0 + 0.001*(x*x + y*y)), 2);
    double avaliacao = 1.0/(0.5 + numerador / denominador); // Inverte para trocar mínimo pelo máximo

    return avaliacao;
}

int giraRoleta(vector<double> &avaliacao, int n, double soma_roleta){
    double aux = 0;
    double rnd = dist1_real(rng) * soma_roleta; // Evita operador / (divisão de ponto flutuante), que é custoso

    for(int i = 0; i < n; i++){
        aux += avaliacao[i];
        if(aux >= rnd) return i;
    }

    // Se o último for o escolhido (rnd = 90, last = 95)
    return n-1;
}

void singlePointCrossover(string &pai1, string &pai2, string &filho1, string &filho2, int n){
    int rnd = dist43(rng);

    for(int i = 0; i < rnd; i++){
        filho1.push_back(pai1[i]);
        filho2.push_back(pai2[i]);
    }
    for(int i = rnd; i < n; i++){
        filho1.push_back(pai2[i]);
        filho2.push_back(pai1[i]);
    }
}

```

```

}

void mutacao(string &filho, int n, double taxa_mutacao){
    for(int i = 0; i < n; i++){
        double rnd = dist1_real(rng);

        if(rnd < taxa_mutacao){
            // Inverte
            if(filho[i] == '1') filho[i] = '0';
            else filho[i] = '1';
        }
    }
}

int main(){
    vector<string> populacao;
    vector<string> filhos;
    vector<double> avaliacao;
    int n = 1000; // Número de indivíduos (deve ser par)
    int n_bits = 44; // Número de bits
    double soma_roleta = 0;
    double aux = 0;
    int epoch = 10;

    // Inicializa população
    for(int i = 0; i < n; i++){
        string cromossomo;
        inicializaElemento(cromossomo, n_bits);
        populacao.push_back(cromossomo);
    }

    for(int i = 0; i < n; i++){ cout << populacao[i] << endl; }

    while(epoch--){
        // Avaliação
        cout << fixed << setprecision(3);
        for(int i = 0; i < n; i++) avaliacao.push_back(calculaAvaliacao(populacao[i]));

        cout << "Geração " << epoch << " - [";
        for(int i = 0; i < n; i++) cout << "(" << binParaDouble(populacao[i], 0, 21) * 0.000023841863594449
        << ", " << binParaDouble(populacao[i], 22, 43) * 0.000023841863594449 << ")", ";";
        cout << "]" << endl;

        // Seleção, Crossover e Mutação
        for(int i = 0; i < n; i++) soma_roleta += avaliacao[i]; // Obter soma_roleta
        for(int i = 0; i < n/2; i++){
            // Seleção
            int pai1 = giraRoleta(avaliacao, n, soma_roleta); // Escolhe índice
            int pai2 = giraRoleta(avaliacao, n, soma_roleta);

            // Crossover
            string filho1, filho2;
            singlePointCrossover(populacao[pai1], populacao[pai2], filho1, filho2, n_bits);

            // Mutação
            mutacao(filho1, n_bits, 0.005);
            mutacao(filho2, n_bits, 0.005);

            filhos.push_back(filho1);
            filhos.push_back(filho2);
        }

        populacao = filhos;

        // Limpa
        filhos.clear();
        avaliacao.clear();
        soma_roleta = 0;
    }
}

```



```
}  
}
```

## Teoria dos GAs

Na teoria dos GAs proposta por John Holland, um **esquema** é um template que representa um subconjunto de indivíduos na população que compartilham similaridades em posições específicas do cromossomo. Para representá-los, adicionamos o caractere curinga `*`. Por exemplo, o esquema `1****1` descreve qualquer indivíduo de 6 bits que comece com `1` e termine com `1` (como `111011`).

Todo esquema possui duas métricas essenciais:

1. **Ordem** ( $o(H)$ ): O número de posições definidas (diferentes de `*`). Ex: `1****1` tem ordem 2.
2. **Tamanho** ( $\delta(H)$ ): A distância física entre o primeiro e o último bit definidos. Ex: `1****1` tem tamanho 4.

De acordo com o Teorema dos Esquemas de Holland, o algoritmo genético progride encontrando pequenos “blocos de construção” (esquemas com alta avaliação e tamanho curto) e combinando-os ao longo das gerações para construir soluções cada vez melhores.

No crossover de um ponto, se tivermos o esquema do pai 1 como sendo `1****1`:

| Tipo do Pai 2       | Filho 1 tem <code>1****1</code> ? | Filho 2 tem <code>1****1</code> ? | Resultado do esquema  |
|---------------------|-----------------------------------|-----------------------------------|-----------------------|
| <code>1****1</code> | Sim                               | Sim                               | Sobreviveu e duplicou |
| <code>0****1</code> | Sim                               | Não                               | Sobreviveu            |
| <code>1****0</code> | Não                               | Sim                               | Sobreviveu            |
| <code>0****0</code> | Não                               | Não                               | Destruído             |

Assim, imagine que temos um cromossomo que desenvolveu no meio a solução perfeita: `*1101*`. Ele conseguiu construir esse bloco/esquema perfeito! Mas, quando cortamos no meio para fazer o crossover de um ponto com outro cromossomo, destruímos esse esquema que foi encontrado.

Assim, o crossover de um ponto não consegue manter todos os esquemas, de modo que esses esquemas não preservados demoram mais gerações para serem preservados.

## Outros operadores genéticos

### Introdução

Agora, vamos dividir melhor os operadores de mutação e crossover. Cada um deles receberá uma porcentagem única, de modo que a soma seja 100%, e para decidir qual operador será aplicado a cada instante rodaremos uma roleta viciada.

A seleção de indivíduos é, obviamente, feita depois da seleção do operador genético a ser aplicado, visto que o operador de mutação requer um indivíduo enquanto o operador de crossover requer dois.

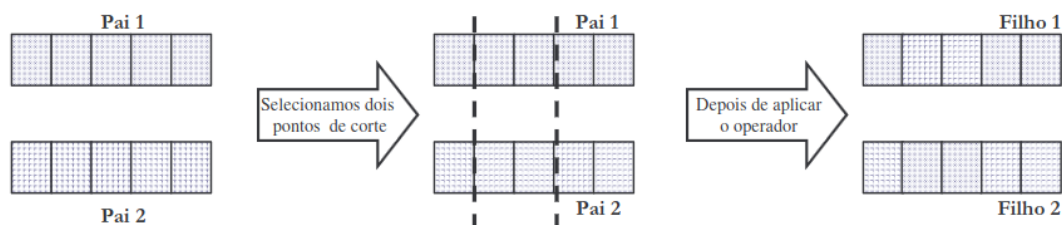
### Crossover de dois pontos

Para melhorar a eficiência do crossover de 2 pontos, podemos introduzir o crossover de 2 pontos. Mas ao invés de sortear 1 ponto de corte, sorteamos dois. O primeiro filho será formado pela parte

do primeiro pai na extremidade e pela parte do segundo pai entre os pontos de corte. O segundo filho será formado pelas partes restantes.

O crossover de 2 pontos reduz a destruição causada pelo crossover de 1 ponto.

Criar 2 filhos a partir de 2 pais significa tentar aproveitar ao máximo todas as partes do cromossomo dos pais vencedores.

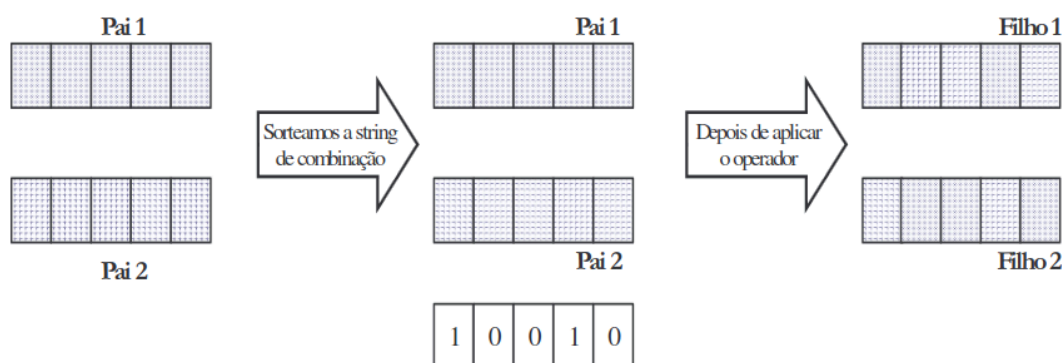


### Crossover uniforme

Apesar do crossover de dois pontos ser capaz de combinar vários esquemas fora da alçada do crossover de um ponto, existem alguns esquemas que ele não consegue manter. Para isso temos o crossover uniforme, que é capaz de combinar todo e qualquer esquema.

Para cada gene, é sorteado o número zero ou um. Se o sorteado for um, o filho 1 recebe o gene do primeiro pai e o filho 2 recebe o gene do segundo pai. Se o sorteado for zero, faz-se o contrário.

Assim, dividimos ao máximo o problema. Sorteia-se cada bit. O objetivo do GA será encontrar o melhor bit para cada posição.

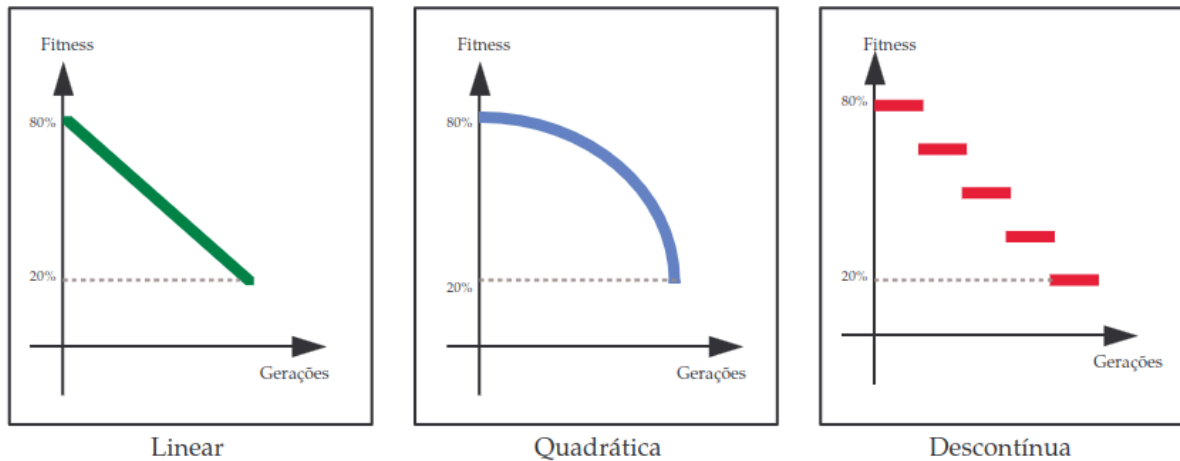


### Operadores com percentagens variáveis

Até agora, quando falamos de crossover e mutação, para selecionar qual iria atuar, atribuíamos uma porcentagem fixa, rodando uma roleta viciada.

Mas no GA, ocorre o seguinte: no início, o ideal é ter muita reprodução, pois a diversidade genética é grande, e queremos explorar o máximo possível o espaço de soluções. Depois de um grande número de rodadas há pouca diversidade genética na população, e seria mais interessante o operador de mutação ser aplicado mais frequentemente do que o operador de crossover, para reinserir diversidade genética na população.

Assim, é ideal a chance de escolha do operador de crossover caísse quando a diversidade diminuísse.



## Operador de mutação dirigida

O problema do operador de mutação simples é que todas as partes do cromossomo têm igual probabilidade de serem modificadas por esse operador, sem distinção. Mas em vários casos o problema pode estar concentrado em um esquema dominante entre as melhores soluções, que não some.

Assim, uma maneira de modificar o esquema dominante é com um novo operador de mutação, que só começa a agir depois de um grande número de gerações. Quando ativado, ele busca as  $n$  melhores soluções dentro da população padrão e verifica qual é a bagagem cromossomial que elas têm em comum. Depois de descobrir o esquema em comum, o operador realiza as mutações dentro desse esquema somente.

Mas há alguns problemas.

1. Ajuste de parâmetros: Se  $n$  for pequeno ou grande demais, pode não se encontrar o esquema dominante. Se o número de rodadas antes de aplicar o operador não for grande o suficiente, o operador é usado sem necessidade. Se for grande demais, muitas rodadas sem valor terão decorrido.
2. Sobrevida das novas soluções: Aplicando mutação sobre o esquema dominante, temos a tendência de gerar soluções ruins para o problema (já que o esquema dominante é o ótimo que o algoritmo achou)
3. Saber quando parar: Quando já tiver diversidade genética o suficiente (limitar indivíduos criados é uma solução)

## Outros módulos de população

### Elitismo

Os  $n$  melhores indivíduos de cada geração não devem “morrer” junto com sua geração, para preservar seus genomas.

### Steady State

Substituição gradual dos piores pais pelos filhos, um a um, a cada rodada. A ideia é imitar a vida real. Decide-se quantas rodadas cada indivíduo vai durar. Se descartarmos rapidamente, a diversidade genética diminui rápido (prendendo em um máximo local), mas se não descartarmos, demora-se mais para convergir.

## Steady State sem duplicatas

O problema do steady state é que há uma convergência muito rápida da população, diminuindo a variedade genética. Para evitar a convergência rápida, verifica-se se o indivíduo gerado é idêntico a algum já presente na população. Se for, ele é descartado.

Isso dá melhores resultados que o Steady State, mas a verificação é pesada computacionalmente.

## Outros tipos de função de avaliação

### Introdução

Há alguns problemas que podem ocorrer com a função de avaliação.

#### Caso 1: Superindivíduo

Podem haver um ou mais indivíduos que são extremamente melhores que os outros da população. Nesse caso, ele será permanentemente selecionado pelo módulo de seleção, causando uma perda imediata da diversidade genética.

#### Caso 2: Pequena diferença entre as fitness

Se a avaliação diferir pouco percentualmente (ex: intervalo [999, 1000]), o algoritmo de GA não consegue perceber que uma pequena diferença pode significar uma grande qualidade na solução.

Ex: 999.001 e 999.999 são 49.97% e 50.13%.

### Normalização linear

1. Ordene os cromossomos em ordem decrescente de valor.
2. Crie novas avaliações para cada um dos indivíduos de forma que o melhor de todos receba um valor fixo ( $k$ ) e os outros recebam valores iguais ao valor do indivíduo imediatamente anterior na lista ordenada menos um valor de decremento constante ( $t$ ).

```
avaliacao[0] = k  
avaliacao[i] = avaliacao[i-1] - t
```

Isso resolve o problema do superindivíduo e da aglomeração das funções de avaliação, mas em contrapartida temos de escolher  $k$  e  $t$  adequados.

### Normalização não linear

Esse método consiste em transformar os valores da avaliação com uma função não linear. Por exemplo, aplicar log resolve o problema do super-indivíduo. Por outro lado, aplicar  $x^n$  pune com mais rigor soluções “ruins”, onde  $n > 1$  para casos de funções de avaliação com módulos maiores que 1 e  $n < 1$  para casos de função de avaliação no intervalo  $[0, 1]$ .

### Windowing

1. Ache o valor mínimo dentre as avaliações da população
2. Dê a cada um dos cromossomos uma avaliação  $avaliacao - \text{minimo}$ .

Ex: 999.001 e 999.999 (49,97% e 50,13%) se tornam 0.001 e 0.999 (0.1% e 99.99%), voltando a fazer uma pressão seletiva em favor do melhor indivíduo.